

Given: Bead on a Parabolic Wire

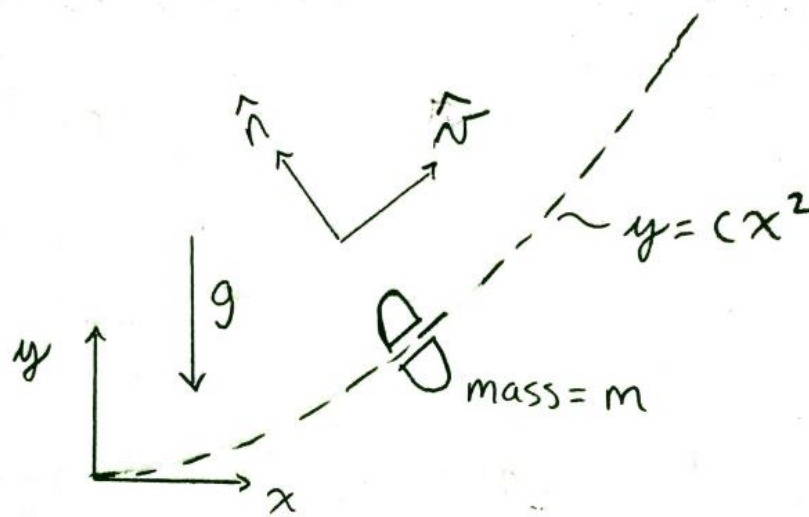


Figure 1 – Diagram of Entire System

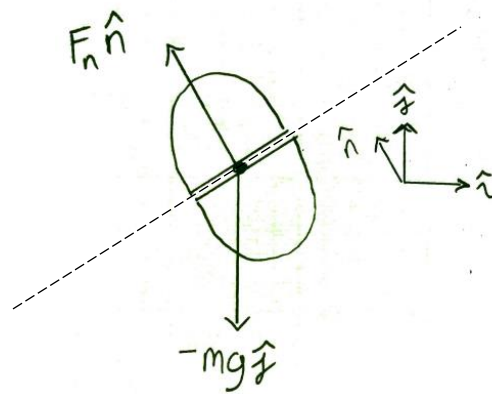


Figure 2 – Free Body Diagram of Bead.

NOTE: $\hat{n}$ is **not** into the page, even though the coordinate frame may be misleading.

Find

- (a) Derive the equations of motion using the x axis as the generalized coordinate.

Assume

- Constant gravity
- No friction
- Bead is a point mass

Step 1: Linear Momentum Balance

$$\begin{aligned} \sum \vec{F} &= m\vec{a} & (1) \\ \swarrow & \quad \searrow \\ \sum \vec{F} &= F_n \hat{n} - mg\hat{j} & m\vec{a} = m(\ddot{x}\hat{i} + \ddot{y}\hat{j}) \end{aligned}$$

$$F_n \hat{n} - mg\hat{j} = m(\ddot{x}\hat{i} + \ddot{y}\hat{j}) \quad (2)$$

Problem: (2) cannot be solved as a system of equations until all vectors are written in terms of $\hat{i}$ and $\hat{j}$.

Solution: Do things the “American” way – dot the entire equation with a vector that is $\perp$ to $\hat{n}$.

Velocity is $\perp$ to $\hat{n}$.

Step 2: Dot Both Sides of the Equation with Velocity

$$\vec{v} \cdot \{F_n \hat{n} - mg\hat{j} = m(\ddot{x}\hat{i} + \ddot{y}\hat{j})\}$$

This “kills” the $\hat{n}$ term. Next, the remaining terms must be dotted with velocity:

$$-mg(\vec{v} \cdot \hat{j}) = m[\vec{v} \cdot (\ddot{x}\hat{i} + \ddot{y}\hat{j})] \quad (3)$$

Definition of Cartesian velocity:

$$\vec{v} = \dot{x}\hat{i} + \dot{y}\hat{j} \quad (4)$$

Substituting:

$$-mg((\dot{x}\hat{i} + \dot{y}\hat{j}) \cdot \hat{j}) = m[(\dot{x}\hat{i} + \dot{y}\hat{j}) \cdot (\ddot{x}\hat{i} + \ddot{y}\hat{j})]$$

Simplifying:

$$-g\dot{y} = \dot{x}\ddot{x} + \dot{y}\ddot{y} \quad (5)$$

Problem: (5) is in terms of x and y . We are finding equations of motion using x as generalized coordinate.

Solution: Use the wire’s curve as a constraint to convert y terms to x .

Step 3: Find $\dot{y}$ and $\ddot{y}$ in terms of x , $\dot{x}$, and $\ddot{x}$

NOTE: In this step, color is used to illustrate substitution in step 5.

From the problem definition:

$$y = cx^2 \quad (6)$$

Differentiate with respect to time (observing the *chain rule*):

$$\dot{y} = 2cx\dot{x} \quad (7)$$

Differentiate with respect to time (observing the *product rule*):

$$\ddot{y} = 2c\dot{x}^2 + 2cx\ddot{x} \quad (8)$$

Step 4: Substitute (7) and (8) into equation of motion (5)

$$-g(2cx\dot{x}) = \dot{x}\ddot{x} + (2cx\dot{x})(2c\dot{x}^2 + 2cx\ddot{x})$$

Simplifying:

$$-2gcx\dot{x} = \dot{x}\ddot{x} + 4c^2x\dot{x}^3 + 4c^2x^2\dot{x}\ddot{x}$$

Cancel $\dot{x}$ terms:

$$-2gcx = \ddot{x} + 4c^2x\dot{x}^2 + 4c^2x^2\ddot{x} \quad (9)$$

Step 5: Solve for $\ddot{x}$

$$\ddot{x} = \frac{-2gcx - 4c^2x\dot{x}^2}{1 + 4c^2x^2} \quad (10)$$

Step 1: Verify Analytical Solution with Symbolic Algebra/Calculus

Code:

```

%% symbolic algebra
syms x x_dot x_d_dot c g m
syms Fn

% ---- STEP 1: Translate diagrams into computer algebra -----
i = [1;0;0]; j = [0;1;0]; k = [0;0;1];

y      = c*x^2;
y_dot  = jacobian(y, [x, x_dot]) * [x_dot; x_d_dot];
y_d_dot = jacobian(y_dot, [x, x_dot]) * [x_dot; x_d_dot];

v = [x_dot; y_dot; 0]; % (4) from solution
a = [x_d_dot; y_d_dot; 0];

% unit vector perpendicular to v (direction of normal force)
e_n = cross(v,k);

% ---- STEP 2: LMB -----
Fsum = (Fn)*e_n - (m*g)*j;
eq2 = Fsum == m*a;

% ---- STEP 3-6: Dot Both Sides of eq2 with v, all in terms of x -----
eq5 = transpose(v) * eq2; % this is already in terms of x
eq9 = eq5 / x_dot; % because there is an x_dot in every term
eq10 = solve(simplify( eq9 ), x_d_dot);

pretty(eq10)

```

Result:

$$\frac{x^2 c^2 x_{\dot{}}^2 + g x^2 c^2}{4 c^2 x^2 + 1}$$

Figure 3 – The result of MATLAB symbolic computation, which agrees with the analytical equation of motion above (10)

Step 2: Numerically Solve Trajectory to Verify Bead Follows Parabola

Physical properties for simulation:

$$m = 1 \text{ kg} \quad g = 1 \text{ m/s}^2 \quad c = 1 \text{ m}^{-1}$$

Initial conditions:

$$x_0 = 1 \text{ m} \quad \dot{x}_0 = 0 \text{ m/s}$$

Code:

```

%% ----- Main Code -----
% initial conditions
x0 = 1;
x_dot0 = 0;
z0 = [x0; x_dot0];

% solver settings
opts.RelTol = 1E-7;
opts.AbsTol = 1E-7;
tf = linspace(0,10,200);

% solver
[t, z_soleom] = ode45(@RHS_EOM, tf, z0, opts, p);

% graph the solution
x_sol = z_soleom(:,1);
y_sol = p.c*x_sol.^2;

plot3(x_sol, y_sol, t)
grid('on')
xlabel('X Axis')
ylabel('Y Axis')
zlabel('Time Increasing')
title({'Bead on a Frictionless Parabolic String',
       'Simulated for 10 Seconds'})

%% ----- Right-Hand-Side Function -----
function z_dot = RHS_EOM(t, z_t, p)

    x = z_t(1);
    x_dot = z_t(2);

    x_d_dot = (-2*p.g*p.c*x - 4*p.c^2*x*x_dot^2) / (1 + 4*p.c^2*x^2);

    z_dot = [x_dot; x_d_dot];

end

```

Figure 4 – Numerical Simulation MATLAB Script for Frictionless Bead on a Parabolic Wire

Result:

Results for the 10 second simulation were compared to the parabola $y = x^2$, and the error was summed across all 200 points:

```
>> test_parabola = p.c * x_sol.^2;  
>> sum(abs(test_parabola - y_sol))  
  
ans =  
  
0
```

Figure 5 – This measures the deviation of the simulated bead from the parabolic wire. It has deviation of zero.

The code in Figure 5 shows that over the 200 points of bead trajectory returned by the simulation, there was a sum total zero meters of deviation from the expected y values at every returned x value.

The script in Figure 5 also returns a graph, which shows the trajectory of the bead over the 10 seconds simulated.

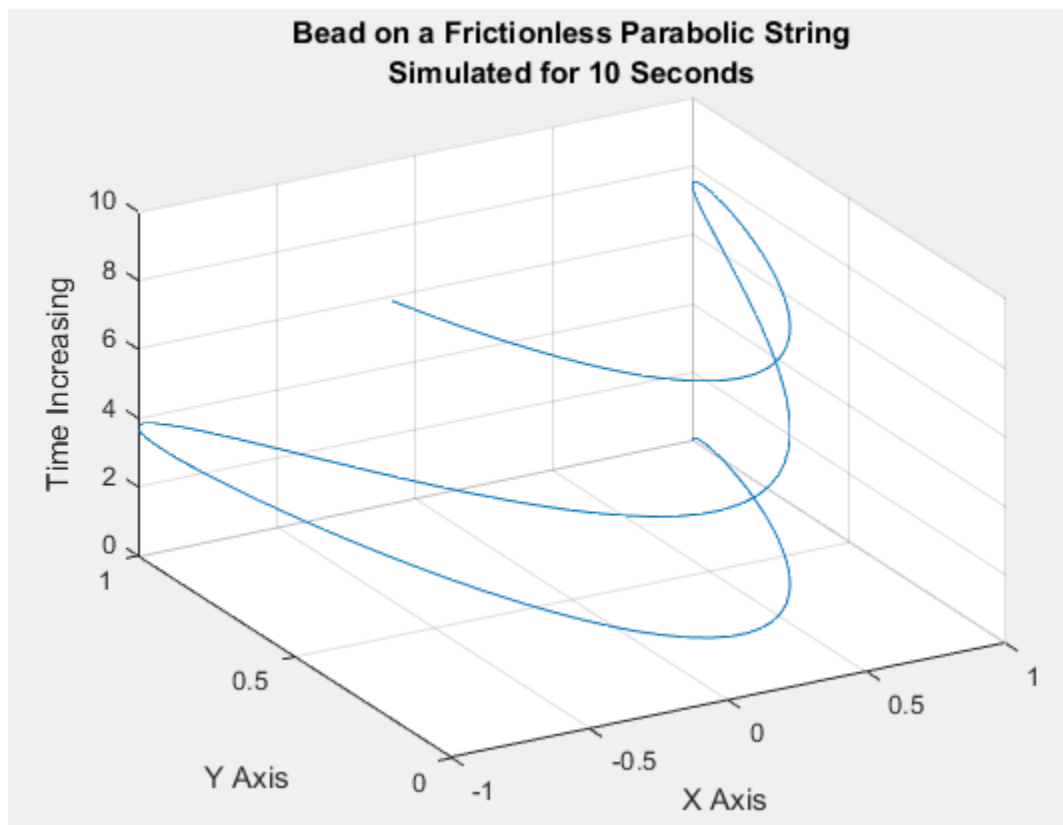


Figure 6 – Trajectory of Bead on a Frictionless Parabolic String, Simulated for 10 Seconds. Time shown as z axis.

Step 3: Verify Errors in Kinetic Energy Consistent with Solver Tolerance

For a good simulation, error in kinetic energy should decrease logarithmically with the solver's tolerance. For this test, gravity was set to zero in order to force constant velocity, and therefore constant kinetic energy.

The follow script was used, simulating the trajectory and recording the final velocity for 10 different tolerance values:

```
%% check kinetic energy error

% set gravity = 0 for this test
p.g = 0;

% set initial conditions for this test
x0 = -1; % m
x_dot0 = 1; % m/s
z0 = [x0; x_dot0];

% run for 10 different, varying tolerances
tols = 1:10;
v_ends = zeros(1,length(tols));
for i = tols

    opts.RelTol = 1*10^-i;
    opts.AbsTol = 1*10^-i;
    tf = linspace(0,10,200);

    % solver
    [t, z_oleom] = ode45(@RHS_EOM, tf, z0, opts, p);

    v_ends(i) = norm(eval(subs(v, [x, x_dot, c], [z_oleom(end,:),
p.c]))));

end

% the kinetic energy at the start is determined by the init. conditions
v_0 = eval(subs(v, [x, x_dot, c], [transpose(z0), p.c]));
v_0 = sqrt(v_0(1).^2 + v_0(2).^2);
Ek_0s = 0.5 * p.m * v_0^2 * ones(1,length(tols));
Ek_ends = 0.5 * p.m * v_ends.^2;
delta_Ek = Ek_ends - Ek_0s;

figure(10)
loglog(1*10.^(-tols), delta_Ek)
title({'Kinetic Energy Error vs Solver Tolerance', 'Bead on a Parabolic Wire', 'Evaluated After 10 Second Simulation'});
ylabel('E_k,end - E_k,start')
xlabel('ode45 Tolerance (RelTol = AbsTol)')
```

Figure 7 – Kinetic Energy Error Validation MATLAB Code

Please see the next page for the resulting plot, which shows a linear relationship. A linear relationship is consistent with the expected behavior.

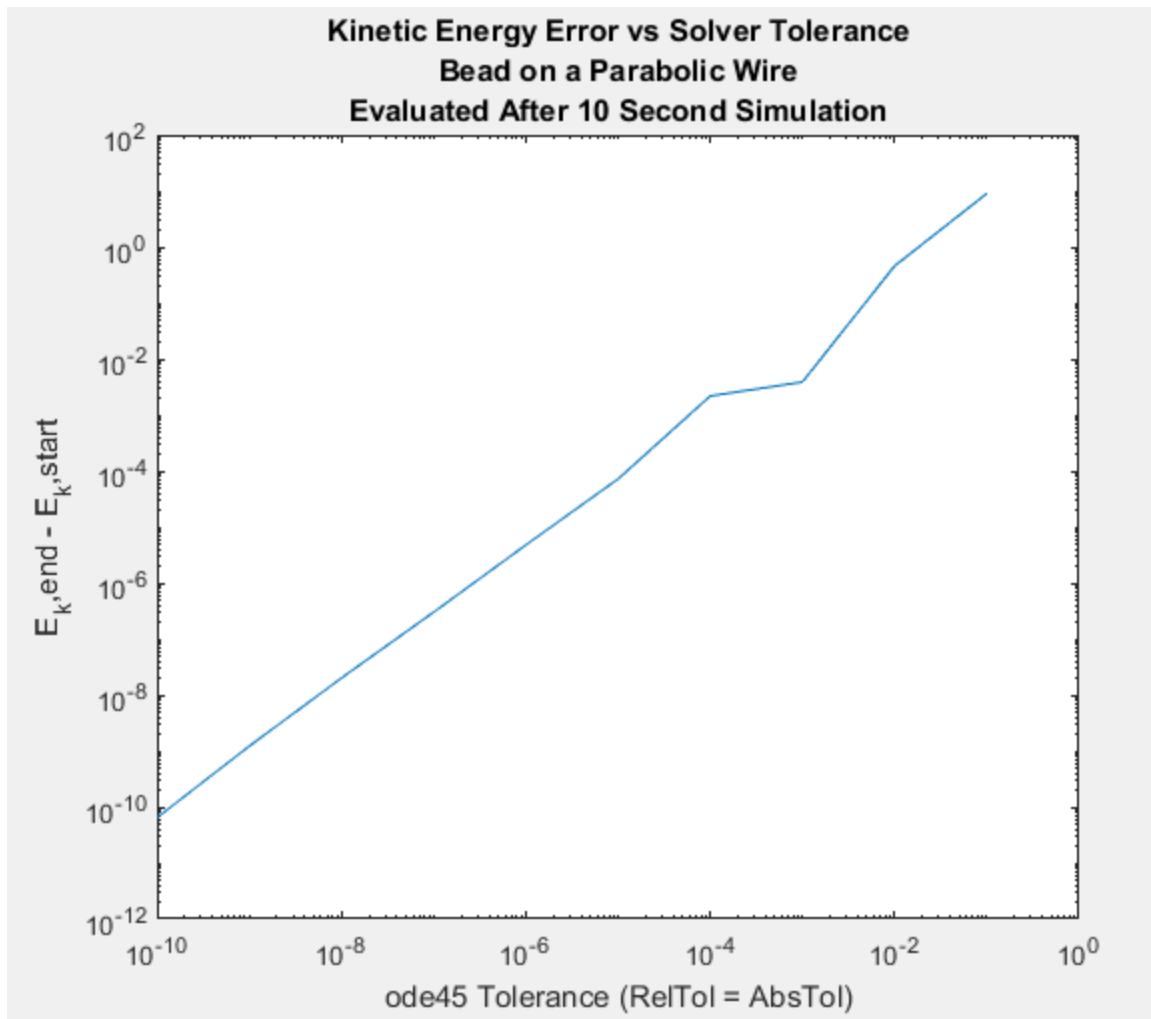


Figure 8 – Plot of kinetic energy error vs ode45 tolerance